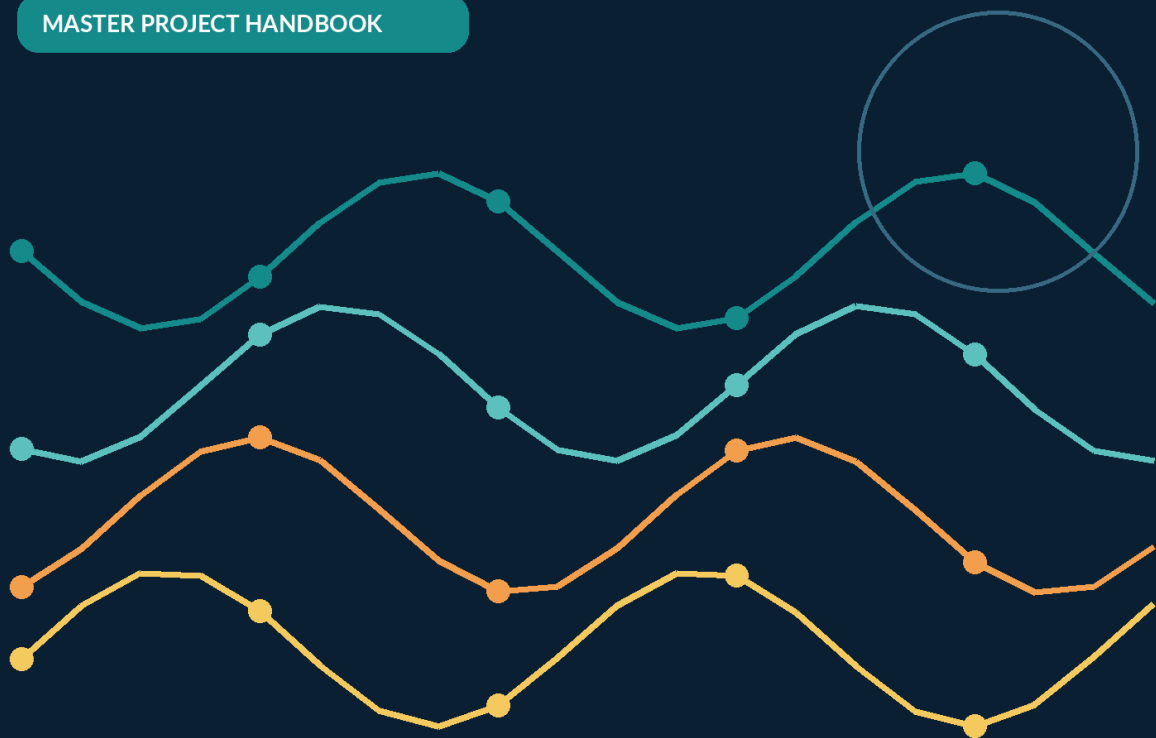


## MASTER PROJECT HANDBOOK



# Time Irreversibility in Cell Motility

Biophysics • Python • Scientific Computing • Reproducible Research

## Complete project record through the end of Phase 3

with repository audit, code listings, educational explanations,  
verified outputs, scientific interpretation, and a Phase 4 handoff

Prepared for Abolfazl Pourhashem

Repository snapshot inspected 26 July 2026

# Document purpose and evidence standard

This handbook is simultaneously a project report, a teaching text, a reproducibility record, and a transfer document. It is designed so that a future collaborator—or a new ChatGPT conversation—can understand what was planned, what was actually executed, what the numerical outputs were, what remains uncertain, and exactly where work should resume.

## Evidence hierarchy used to build this handbook

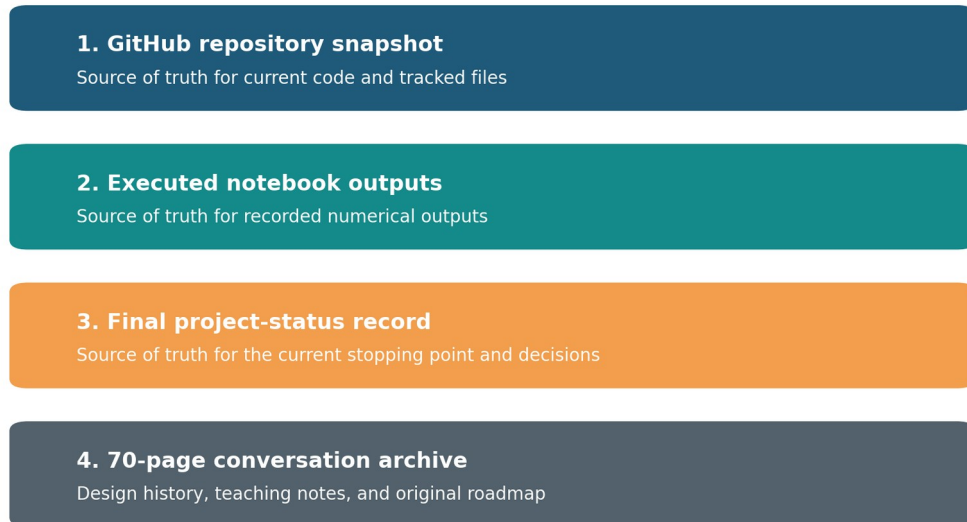


Figure 1. Evidence hierarchy used to reconcile project history, notebook outputs, and the live repository.

### Evidence labels used throughout

REPOSITORY-VERIFIED means the item exists in the current GitHub main branch. NOTEBOOK-OUTPUT VERIFIED means the value is stored in an executed notebook output. PROJECT-RECORD REPORTED means it is documented in the final status record but is not present in the public repository. PLANNED means it has not yet been executed.

## Project Handoff Brief — Read This First

### Exact stopping point

Phase 3 core implementation is complete. The next active task is Phase 4: validate whether the common directed motion is imaging/stage drift or real collective migration, then perform a defensible coordinate correction and rebuild displacement steps. Do not compute or interpret a final irreversibility metric before this validation.

| Item                   | Current verified state                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------|
| Repository             | Public GitHub repository; main branch; six commits visible at inspection time.                    |
| Dataset                | Cell Tracking Challenge Fluo-C2DL-MSD, sequence 01.                                               |
| Calibration            | 0.3 $\mu\text{m}/\text{pixel}$ ; 20 min/frame; 48 frames.                                         |
| Tracking               | Fiji/TrackMate: LoG detector + Simple LAP tracker; exact settings recorded below.                 |
| Clean trajectory table | 474 tracked detections, 35 tracks, canonical columns <code>cell_id/frame/t_min/x_um/y_um</code> . |
| Phase 3 output         | 1,187 total rows across $\tau = 1, 2, 4$ .                                                        |
| Next step              | Create <code>04_drift_correction_MSC01.ipynb</code> and validate/correct common motion.           |

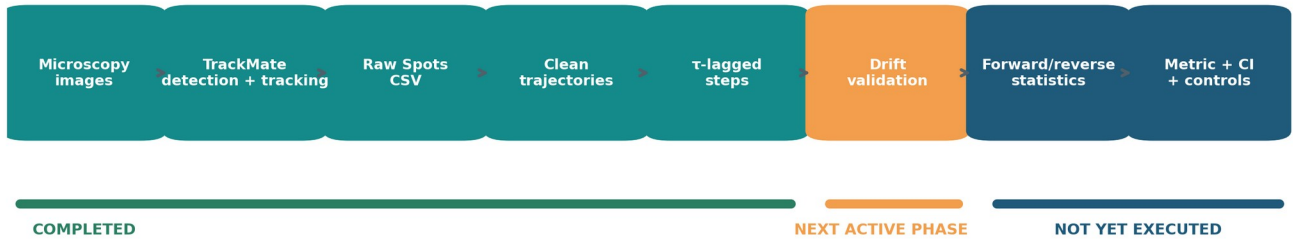


Figure 2. End-to-end project pipeline and current position.

Repository link: <https://github.com/AbolDpirate/cell-irreversibility>

[↑ Back to Contents](#)

# Contents

---

Every entry below is an internal hyperlink. The document uses static links so navigation works without updating Word fields.

## **1. Scientific Motivation and Scope**

- [1.1 The biological system](#)
- [1.2 Time reversal and non-equilibrium reasoning](#)
- [1.3 The updated interpretation of the original metric](#)

## **2. Roadmap: Original Plan vs. Executed Work**

- [2.1 Original phases 0–11](#)
- [2.2 Updated phase structure](#)

## **3. Reproducible Project Architecture**

- [3.1 Repository snapshot](#)
- [3.2 Conda environment](#)
- [3.3 Python packages, modules, notebooks, and tests](#)
- [3.4 Git and ignored artifacts](#)

## **4. Dataset and Image-Tracking Workflow**

- [4.1 Dataset selection and metadata](#)
- [4.2 Fiji calibration](#)
- [4.3 TrackMate detection and tracking](#)
- [4.4 Raw CSV export](#)

## **5. Phase 2B – Cleaning the TrackMate Spots Table**

- [5.1 Raw table anatomy](#)
- [5.2 Exact cleaning code](#)
- [5.3 Verified outputs](#)
- [5.4 Python learning module](#)

## **6. Mathematics of Trajectories and $\tau$ -Lagged Steps**

- [6.1 Trajectory and displacement definitions](#)
- [6.2 Units and lag selection](#)
- [6.3 What a one-step distribution can and cannot prove](#)

## **7. The Reusable Phase 3 Module: `src/steps.py`**

- [7.1 Full source listing](#)
- [7.2 `validate\_tracks`](#)
- [7.3 `compute\_steps\_for\_tau`](#)
- [7.4 Multiple lags and summaries](#)
- [7.5 Python learning module](#)

## **8. Phase 3 Notebook and Verified Numerical Results**

- [8.1 Exact notebook code](#)
- [8.2 Step-support summary](#)
- [8.3 Descriptive statistics](#)
- [8.4 Drift diagnostic](#)
- [8.5 Saved artifacts](#)

## **9. Reproducibility and Scientific Audit**

- [9.1 What is reproducible now](#)
- [9.2 Repository gaps and mismatches](#)
- [9.3 Algorithmic issues to resolve](#)
- [9.4 Test coverage](#)

## **10. Exact Current Status**

[10.1 Completed](#)

[10.2 Not completed](#)

[10.3 Current scientific question](#)

## **11. Updated Roadmap from Phase 4 to Publication**

[11.1 Phase 4 drift validation](#)

[11.2 Displacement asymmetry](#)

[11.3 Proper temporal irreversibility](#)

[11.4 CI, controls, robustness, thermodynamics](#)

## **12. Resume Protocol for the Next Work Session**

[12.1 Environment and kernel checks](#)

[12.2 Re-run Phase 3](#)

[12.3 Begin Phase 4](#)

[12.4 Git checkpoint](#)

## **Appendix A. Full Supporting Files**

## **Appendix B. Data Dictionaries and Artifact Ledger**

## **Appendix C. Python and Biophysics Glossary**

## **Appendix D. Ready-to-Paste New-Chat Handoff Prompt**

## **Appendix E. References and Source Snapshot**

# 1. Scientific Motivation and Scope

## 1.1 The biological system

Living cells are open, active systems. They consume chemical free energy, remodel their cytoskeleton, generate forces, change shape, and migrate. Those processes motivate a non-equilibrium analysis: can the recorded motion itself reveal statistical signatures that distinguish the observed direction of time from a reversed description?

- The observable is a two-dimensional cell-centroid trajectory extracted from time-lapse microscopy.
- The project is in silico: no new wet-lab experiment is required for the current stage.
- The analysis must distinguish biological motion from tracking artifacts and image/stage drift.
- Any entropy-production interpretation must be stated as a bound or coarse-grained proxy under explicit assumptions—not as the full thermodynamic dissipation of a cell.

## 1.2 Time reversal and non-equilibrium reasoning

At equilibrium, detailed balance implies a symmetry between appropriately defined forward and reverse path statistics. Away from equilibrium, the logarithm of a forward-path probability divided by its reverse-path probability is a central stochastic-thermodynamic quantity. The word path is crucial: a time-ordered sequence generally contains more information than a single displacement.

$$\text{Path-level irreversibility: } \log [ P(\text{path}) / P(\text{time-reversed path}) ]$$

### Interpretation rule

A difference between forward and reversed path probabilities can support time-reversal symmetry breaking. A difference between  $p(\Delta)$  and  $p(-\Delta)$  only establishes inversion asymmetry of single displacements unless extra assumptions connect it to path irreversibility.

## 1.3 The updated interpretation of the original metric

The original project plan proposed an index of the form below. It remains useful as an exploratory displacement-asymmetry statistic, but it should not automatically be labeled a complete temporal-irreversibility measure.

$$I\tau = \langle \log [ p(\Delta\tau) / p(-\Delta\tau) ] \rangle$$

| Analysis level                            | What it directly measures                                      | Appropriate interpretation                                                |
|-------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------|
| Single displacement                       | Whether $(dx, dy)$ and $(-dx, -dy)$ have the same distribution | Directional bias / displacement inversion asymmetry                       |
| Two or more ordered steps                 | Whether short forward sequences resemble reversed sequences    | Temporal ordering asymmetry                                               |
| Full or sufficiently rich path statistics | Forward vs. reversed path probability ratio                    | Time irreversibility; possible entropy-production bound under assumptions |

[↑ Back to Contents](#)

# 2. Roadmap: Original Plan vs. Executed Work

## 2.1 Original phases 0–11

| Original phase | Purpose                                                                                     | Status at this snapshot                           |
|----------------|---------------------------------------------------------------------------------------------|---------------------------------------------------|
| 0              | Conceptual orientation: equilibrium, time reversal, KDE, bootstrap, entropy production, TUR | Conceptual work completed; interpretation updated |
| 1              | Environment and project skeleton                                                            | Completed                                         |
| 2              | Data import / tracking / standard trajectory CSV                                            | Completed for sequence 01                         |
| 3              | Preprocessing and displacement steps                                                        | Core implementation completed; drift              |

| Original phase | Purpose                            | Status at this snapshot                          |
|----------------|------------------------------------|--------------------------------------------------|
|                |                                    | flagged                                          |
| 4              | Density estimation                 | Not executed                                     |
| 5              | Irreversibility metric + bootstrap | Not executed                                     |
| 6              | Time-reversal and shuffle controls | Not executed                                     |
| 7              | Robustness                         | Not executed                                     |
| 8              | Entropy-production/TUR extension   | Not executed                                     |
| 9              | Publication figures                | Not executed                                     |
| 10             | README, Methods, one-page report   | README exists; full publication kit not complete |
| 11             | Optional machine learning          | Not started                                      |

## 2.2 Updated phase structure

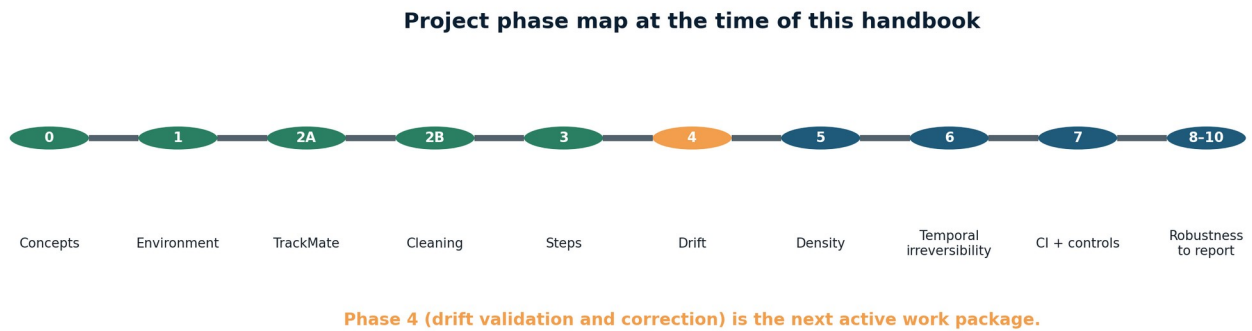


Figure 3. Updated project phases after separating drift validation from density and irreversibility analysis.

The updated roadmap treats drift validation as an independent gate. This prevents a microscope or registration artifact from being mistaken for biological directionality and prevents true collective migration from being removed without evidence.

[↑ Back to Contents](#)

## 3. Reproducible Project Architecture

### 3.1 Repository snapshot

The GitHub main branch inspected on 26 July 2026 contains the following tracked structure. This is the repository-verified tree—not the larger local working tree reported in the project notes.

#### Repository-verified tree

```
cell-irreversibility/
├── data/
│   ├── .keep
│   └── notebooks/
│       ├── 02_clean_spots_MSC01.ipynb
│       ├── 03_compute_steps_MSC01.ipynb
│       └── _path.py
├── src/
│   ├── __init__.py
│   ├── density.py
│   ├── io.py
│   ├── metrics.py
│   ├── plots.py
│   └── steps.py
├── tests/
│   ├── conftest.py
│   └── test_imports.py
├── .gitignore
└── README.md
```

```
└─ env.yml
   instructions.docx
```

### Important repository fact

The CSV inputs and outputs are intentionally ignored by .gitignore and are not present in the public repository. The figures directory is also absent from the tracked snapshot because empty ignored directories are not committed.

## 3.2 Conda environment

Repository file: env.yml

```
name: cell-irreversibility
channels:
  - defaults
dependencies:
  - python=3.11
  - matplotlib
  - pytest
  - numpy
  - pandas
  - scikit-learn
  - jupyterlab
  - ipykernel
```

The environment name is cell-irreversibility and the declared Python version is 3.11. The environment includes pandas for tables, NumPy for arrays and numerical work, Matplotlib for plots, scikit-learn for later statistical/ML extensions, JupyterLab and ipykernel for notebooks, and pytest for tests.

Environment creation and kernel registration

```
conda env create -f env.yml
conda activate cell-irreversibility
python -m ipykernel install --user --name cell-irreversibility --display-name "Python (cell-irreversibility)"
```

## 3.3 Python packages, modules, notebooks, and tests

| Component             | Role                                                        | Current state                             |
|-----------------------|-------------------------------------------------------------|-------------------------------------------|
| src/steps.py          | Validated, reusable $\tau$ -lagged displacement computation | Implemented                               |
| src/io.py             | Future input/validation helpers                             | Placeholder only                          |
| src/density.py        | Future histogram/KDE functions                              | Placeholder only                          |
| src/metrics.py        | Future asymmetry, controls, bootstrap                       | Placeholder only                          |
| src/plots.py          | Future figure helpers                                       | Placeholder only                          |
| notebooks/02...       | TrackMate CSV cleaning                                      | Executed outputs stored                   |
| notebooks/03...       | Displacement steps and drift diagnostics                    | Executed outputs stored                   |
| tests/test_imports.py | Smoke test for imports                                      | Implemented; no scientific unit tests yet |

A module is a .py file. A package is a directory that Python treats as a collection of modules; src/\_\_init\_\_.py marks src as a package. A notebook is an interactive execution document. The project uses notebooks for transparent, stepwise analysis and src/ for reusable logic.

## 3.4 Git and ignored artifacts

Repository file: .gitignore

```
__pycache__/
.ipynb_checkpoints/
.env
.venv
*.pyc
.DS_Store
data/*
!data/.keep
```



```
figures/*
*.mp4
```

Ignoring data can be appropriate when raw images or derived tables are large, licensed, or easily regenerated. However, reproducibility then requires a documented download step, checksums or dataset version, and executable scripts that recreate every derived artifact. At present, the public repository does not contain enough data artifacts to run the notebooks end-to-end without the local working directory.

[↑ Back to Contents](#)

## 4. Dataset and Image-Tracking Workflow

### 4.1 Dataset selection and metadata

| Field               | Recorded value                                                                                                                                                               |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset             | Fluo-C2DL-MSK from the Cell Tracking Challenge                                                                                                                               |
| Sequence            | 01                                                                                                                                                                           |
| Imaging             | 2D fluorescence time-lapse                                                                                                                                                   |
| Biological material | Mesenchymal stem cells (the project record describes human MSC; the CTC benchmark paper labels the dataset as rat MSC—this metadata should be reconciled before publication) |
| Frames              | 48                                                                                                                                                                           |
| Pixel size          | 0.3 $\mu\text{m}$ /pixel                                                                                                                                                     |
| Frame interval      | 20 min/frame                                                                                                                                                                 |

#### Metadata discrepancy to resolve

The project conversation described human mesenchymal stem cells, whereas the Cell Tracking Challenge benchmark literature identifies Fluo-C2DL-MSK as rat mesenchymal stem cells. The final Methods section must use the authoritative dataset metadata.

### 4.2 Fiji calibration

1. Open sequence 01 as an image sequence/stack in Fiji.
2. Convert the stack interpretation to Channels = 1, Slices = 1, Frames = 48.
3. Set pixel width and height to 0.3  $\mu\text{m}$ .
4. Set the frame interval to 20 minutes.
5. Confirm that TrackMate reads spatial and temporal calibration before detection.

## 4.3 TrackMate detection and tracking



No additional spot filters • No track filters • Median filter ON • Sub-pixel localization ON

Figure 4. Recorded Fiji/TrackMate workflow and parameters.

| Stage         | Parameter                 | Value              |
|---------------|---------------------------|--------------------|
| Detector      | Algorithm                 | LoG detector       |
| Detector      | Estimated object diameter | 28 $\mu\text{m}$   |
| Detector      | Quality threshold         | 2                  |
| Detector      | Median filter             | Enabled            |
| Detector      | Sub-pixel localization    | Enabled            |
| Spot filters  | Additional filters        | None               |
| Tracker       | Algorithm                 | Simple LAP tracker |
| Tracker       | Linking max distance      | 8 $\mu\text{m}$    |
| Tracker       | Gap-closing max distance  | 15 $\mu\text{m}$   |
| Tracker       | Maximum frame gap         | 2 frames           |
| Track filters | Additional filters        | None               |

The Laplacian-of-Gaussian detector identifies approximately blob-like bright objects at a scale related to the estimated diameter. The Simple LAP tracker links detections across frames by solving an assignment problem subject to distance and gap-closing constraints. These parameters are not universal biological constants; they are analysis choices that should eventually be validated against visual inspection or reference annotations.

## 4.4 Raw CSV export

The final raw table was exported from TrackMate's Spots table rather than the MotilityLab export. Each row represents one detection at one time point and contains trajectory identifiers, coordinates, time, quality, radius, and intensity features.

Locally reported raw input path (not present in GitHub)

```
data/raw/Fluo-C2DL-MSC/01/MS01_allspots.csv
```

[↑ Back to Contents](#)

## 5. Phase 2B – Cleaning the TrackMate Spots Table

### 5.1 Raw table anatomy

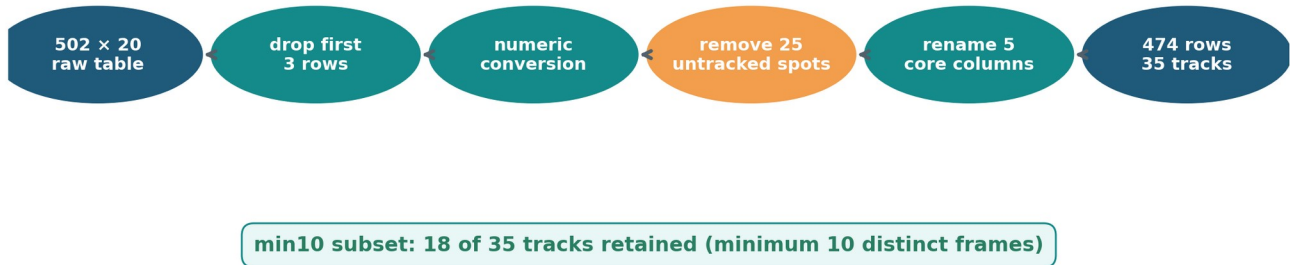


Figure 5. Data reduction from raw TrackMate export to canonical trajectories.

| Checkpoint                          | Verified output                             |
|-------------------------------------|---------------------------------------------|
| Raw pandas table                    | 502 rows × 20 columns                       |
| After removing 3 metadata/unit rows | 499 rows × 20 columns                       |
| Rows with missing TRACK_ID          | 25                                          |
| Tracked detections retained         | 474 rows                                    |
| Unique tracks                       | 35                                          |
| Track length distribution           | mean 13.54; median 10; min 2; max 48 frames |
| min10 subset                        | 18 of 35 tracks                             |

The three first rows were not observations: TrackMate embedded labels and units inside the table. Their presence forced many numeric columns to be read as object/string dtype. Cleaning therefore required both row removal and explicit numeric conversion.

### 5.2 Exact cleaning code

#### Reconstructed cell order from the repository notebook

```

# Cell 1 – Load the raw TrackMate Spots CSV
import pandas as pd

raw_path = "../data/raw/Fluo-C2DL-MS/01/MS01_allspots.csv"
df = pd.read_csv(raw_path)

df.shape, df.columns
print(df.head) # Current repository cell prints the method object, not df.head()

# Cell 2 – Remove TrackMate's three non-data header/unit rows
spots = df.iloc[3:].copy()
print(spots.shape)
spots.head()

# Cell 3 – Convert important columns from object/string to numeric types
num_cols = [
    "ID", "TRACK_ID", "QUALITY",
    "POSITION_X", "POSITION_Y", "POSITION_Z",
    "POSITION_T", "FRAME", "RADIUS", "VISIBILITY"
]

for c in num_cols:

```

```

spots[c] = pd.to_numeric(spots[c], errors="coerce")

spots[num_cols].dtypes

# Cell 4 – Count detections without a TrackMate TRACK_ID
spots["TRACK_ID"].isna().sum()

# Cell 5 – Keep only detections that belong to a trajectory
spots = spots[spots["TRACK_ID"].notna()].copy()
spots.shape

# Cell 6 – Select and rename the canonical trajectory columns
cols_map = {
    "TRACK_ID": "cell_id",
    "FRAME": "frame",
    "POSITION_T": "t_min",
    "POSITION_X": "x_um",
    "POSITION_Y": "y_um",
}

tracks = spots[list(cols_map.keys())].rename(columns=cols_map)
print(tracks.head())
tracks.dtypes

# Cell 7 – Sort and enforce integer identifiers/frame indices
tracks = tracks.sort_values(["cell_id", "frame"]).reset_index(drop=True)
tracks["cell_id"] = tracks["cell_id"].astype(int)
tracks["frame"] = tracks["frame"].astype(int)
print(tracks.dtypes)
tracks.head()

# Cell 8 – Measure track lengths
lengths = tracks.groupby("cell_id")["frame"].nunique()
lengths.describe()

# Cell 9 – Create the min10 subset
min_len = 10
good_ids = lengths[lengths >= min_len].index
tracks_filtered = tracks[tracks["cell_id"].isin(good_ids)].copy()
tracks_filtered = tracks_filtered.sort_values(["cell_id", "frame"]).reset_index(drop=True)
tracks_filtered["cell_id"].nunique(), tracks["cell_id"].nunique()

# Cell 10 – Intended output paths
out_path1 = "../data/MS01_tracks_clean.csv"
out_path2 = "../data/MS01_tracks_clean_min10.csv"

# NOTE: these two lines are commented in the current GitHub notebook.
# tracks.to_csv(out_path1, index=False)
# tracks_filtered.to_csv(out_path2, index=False)

```

#### Current notebook bug

The first cell contains `print(df.head)`, which prints a bound method object. The intended inspection call is `df.head()` or `print(df.head())`.

#### Current reproducibility gap

The two `to_csv` lines in the GitHub notebook are commented. A clean “Restart Kernel and Run All” will not recreate `MS01_tracks_clean.csv` or `MS01_tracks_clean_min10.csv` unless those lines are uncommented or another script generates the files.

## 5.3 Verified outputs

| Canonical column | Meaning                           | Expected dtype | Unit          |
|------------------|-----------------------------------|----------------|---------------|
| cell_id          | TrackMate trajectory identifier   | int            | dimensionless |
| frame            | Image frame index                 | int            | frame         |
| t_min            | Acquisition time                  | float          | minutes       |
| x_um             | x-coordinate of detected centroid | float          | μm            |
| y_um             | y-coordinate of detected centroid | float          | μm            |

The full clean table contained 474 rows. The min10 table retained only trajectories with at least 10 distinct frames. This threshold is an implemented convenience, not yet a scientifically optimized inclusion criterion. The earlier planning document mentioned a 30-frame threshold, but the executed notebook used 10.

## 5.4 Python learning module

| Python object/concept | Project example              | Why it was used                                                     |
|-----------------------|------------------------------|---------------------------------------------------------------------|
| DataFrame             | df, spots, tracks            | A labeled two-dimensional table similar to a spreadsheet            |
| Series                | tracks["cell_id"]            | One labeled column; supports vectorized operations                  |
| list                  | num_cols                     | Ordered, mutable collection of column names                         |
| dictionary            | cols_map                     | Maps old column names to new names as key-value pairs               |
| set                   | required columns in steps.py | Unordered unique values; supports set difference for validation     |
| tuple                 | tracks.shape → (474, 5)      | Fixed-size ordered return value; convenient for paired dimensions   |
| for loop              | for c in num_cols            | Repeat numeric conversion for each selected column                  |
| boolean mask          | spots["TRACK_ID"].notna()    | Select rows satisfying a condition                                  |
| copy()                | df.iloc[3:].copy()           | Create an independent object and avoid ambiguous chained assignment |
| NaN                   | missing TRACK_ID             | Standard missing-value marker in pandas/NumPy                       |

A list is ordered and mutable; a tuple is ordered but immutable; a set is unordered and stores unique hashable elements; a dictionary stores key-value mappings. The choice is semantic: num\_cols must preserve an explicit sequence, cols\_map must associate each old name with a new name, and REQUIRED\_TRACK\_COLS must support fast membership and set subtraction.

[↑ Back to Contents](#)

# 6. Mathematics of Trajectories and $\tau$ -Lagged Steps

## 6.1 Trajectory and displacement definitions

For one cell  $i$ , the trajectory is the ordered sequence  $r_i(t) = (x_i(t), y_i(t))$ . The analysis transforms absolute positions into increments over a selected lag  $\tau$ .

$$\Delta r(t) = r(t + \tau) - r(t) = (x(t + \tau) - x(t), y(t + \tau) - y(t))$$

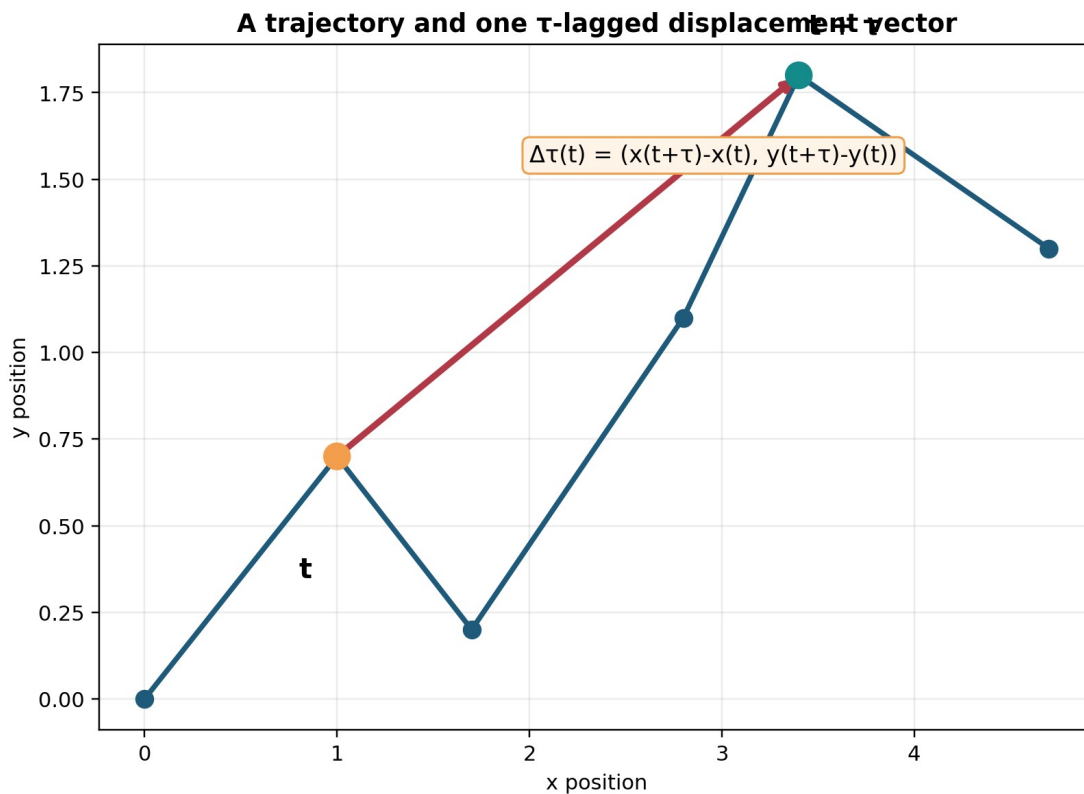


Figure 6. Geometric meaning of a  $\tau$ -lagged displacement vector.

Absolute position depends on the arbitrary image origin. Displacements describe motion over a time interval and are therefore more directly related to velocity-like behavior, directional bias, persistence, and stochastic increments.

## 6.2 Units and lag selection

| tau_frames | Nominal elapsed time | Reason retained                                           |
|------------|----------------------|-----------------------------------------------------------|
| 1          | 20 min               | Maximum temporal resolution and largest sample            |
| 2          | 40 min               | Intermediate scale                                        |
| 4          | 80 min               | Longer-scale motion while retaining 26 contributing cells |

The selected lags were 1, 2, and 4 frames. Larger lags such as 5 or 10 were avoided because short trajectories rapidly stop contributing, reducing both sample size and the number of independent cells.

## 6.3 What a one-step distribution can and cannot prove

A stationary active process may have a symmetric one-step increment distribution while still exhibiting irreversible ordering across multiple steps. For example, cyclic dynamics can look symmetric when projected onto a single increment but asymmetric in ordered pairs. Therefore, the project now separates displacement inversion asymmetry from proper temporal irreversibility.

### Naming convention

Until sequence-level forward/reverse statistics are implemented, use “displacement inversion asymmetry” or “single-step directional asymmetry,” not an unqualified claim of entropy production or full time irreversibility.

[↑ Back to Contents](#)

## 7. The Reusable Phase 3 Module: src/steps.py

### 7.1 Full source listing

Repository-verified source: [src/steps.py](#)

```
# src/steps.py

from __future__ import annotations
import pandas as pd

REQUIRED_TRACK_COLS = {"cell_id", "frame", "t_min", "x_um", "y_um"}

def validate_tracks(tracks: pd.DataFrame) -> None:
    """
    Validate that the input tracks DataFrame has the required columns.
    Raises ValueError with a clear message if something is missing.
    """
    missing = REQUIRED_TRACK_COLS - set(tracks.columns)
    if missing:
        raise ValueError(f"tracks is missing required columns: {sorted(missing)}")

def compute_steps_for_tau(tracks: pd.DataFrame, tau_frames: int) -> pd.DataFrame:
    """
    Compute displacement steps  $\Delta$  for a given lag tau_frames.

    Input 'tracks' must have columns:
        cell_id (int), frame (int), t_min (float), x_um (float), y_um (float)

    Returns a DataFrame with one row per step:
        cell_id, frame_start, t_start_min, frame_end, t_end_min, dx_um, dy_um,
        tau_frames, tau_min
    """
    validate_tracks(tracks)
    if tau_frames < 1:
        raise ValueError("tau_frames must be >= 1")

    # Sort to ensure correct temporal order inside each cell_id
    tracks_sorted = (
        tracks.sort_values(["cell_id", "frame"])
        .reset_index(drop=True)
        .copy()
    )

    # Group by trajectory (cell_id) and use shift to align t+tau next to t
    g = tracks_sorted.groupby("cell_id", group_keys=False)
    x_future = g["x_um"].shift(-tau_frames)
    y_future = g["y_um"].shift(-tau_frames)
    t_future = g["t_min"].shift(-tau_frames)
    f_future = g["frame"].shift(-tau_frames)

    # Displacements
    dx = x_future - tracks_sorted["x_um"]
    dy = y_future - tracks_sorted["y_um"]
    steps = pd.DataFrame({
        "cell_id": tracks_sorted["cell_id"].astype(int),
        "frame_start": tracks_sorted["frame"].astype(int),
        "t_start_min": tracks_sorted["t_min"].astype(float),
        "frame_end": f_future,
        "t_end_min": t_future,
        "dx_um": dx,
        "dy_um": dy,
        "tau_frames": int(tau_frames),
    })

    # tau in minutes (computed from timestamps, robust if time step is not constant)
    steps["tau_min"] = steps["t_end_min"] - steps["t_start_min"]
    # Remove rows where the future point does not exist (end of each track)
    steps = steps.dropna(subset=["dx_um", "dy_um", "frame_end", "t_end_min"]).copy()

    # Fix dtypes after dropna
    steps["frame_end"] = steps["frame_end"].astype(int)
    steps["t_end_min"] = steps["t_end_min"].astype(float)
```

```

return steps

def compute_steps_multi_tau(tracks: pd.DataFrame, taus: list[int]) -> pd.DataFrame:
    """
    Compute steps for multiple taus and concatenate into one DataFrame.
    The output includes 'tau_frames' and 'tau_min' so you can filter later.
    """
    validate_tracks(tracks)

    all_steps = []
    for tau in taus:
        s = compute_steps_for_tau(tracks, tau_frames=tau)
        all_steps.append(s)

    steps_all = pd.concat(all_steps, ignore_index=True)
    return steps_all

def summary_by_tau(steps_all: pd.DataFrame) -> pd.DataFrame:
    """
    Create a compact summary table per tau:
    - number of steps
    - number of contributing cell_ids
    - median tau_min
    """
    summary = (steps_all.groupby("tau_frames")
               .agg(
                   n_steps=("dx_um", "size"),
                   n_cells=("cell_id", "nunique"),
                   median_tau_min=("tau_min", "median"),
               )
               .reset_index())

    return summary

```

## 7.2 validate\_tracks

`validate_tracks` protects downstream code from silently operating on a table with the wrong schema.

`REQUIRED_TRACK_COLS` is a set. Subtracting the set of actual columns returns exactly the required names that are absent.

```

missing = REQUIRED_TRACK_COLS - set(tracks.columns)
if missing:
    raise ValueError(f"tracks is missing required columns: {sorted(missing)}")

```

`raise` intentionally stops execution. `ValueError` communicates that the object exists but its value or structure is inappropriate. The f-string inserts the sorted missing-column list into a human-readable error message.

## 7.3 compute\_steps\_for\_tau

6. Validate the input schema and reject non-positive lags.
7. Sort by `cell_id` and frame so time order is deterministic.
8. Group rows by `cell_id` so no displacement crosses from one cell to another.
9. Use `shift(-tau_frames)` to align future rows beside current rows.
10. Subtract current positions from future positions.
11. Construct a new DataFrame with identifiers, start/end coordinates in time, displacement components, and lag metadata.
12. Drop rows at each track end where a future observation does not exist.
13. Restore integer dtype for `frame_end` after NaN removal.

### Subtle algorithmic issue

`shift(-tau_frames)` advances by rows, not by the numerical frame index. If a trajectory has missing frames, a row labeled `tau_frames = 1` may span more than one actual frame. `tau_min` records the elapsed time, but the categorical label can be misleading. Before Phase 4, test `frame_end - frame_start` and decide whether to enforce exact frame lags using a self-merge on `cell_id` and frame.



## 7.4 Multiple lags and summaries

`compute_steps_multi_tau` creates one DataFrame per lag, stores them in a list, and concatenates them vertically. `summary_by_tau` then groups the combined table by `tau_frames` and calculates named aggregations.

```
n_steps=("dx_um", "size")
n_cells=("cell_id", "nunique")
median_tau_min=("tau_min", "median")
```

Named aggregation makes the output columns explicit. `size` counts rows, `nunique` counts distinct cell identifiers, and `median` summarizes elapsed time robustly. The median alone does not prove that every step has the nominal lag; the full distribution of `tau_min` and `frame_end - frame_start` should be inspected.

## 7.5 Python learning module

| Concept                                         | Meaning in this module                                                                                                    |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Type hint                                       | tracks: <code>pd.DataFrame</code> documents the expected type; <code>-&gt; pd.DataFrame</code> documents the return type. |
| <code>from __future__ import annotations</code> | Defers evaluation of type annotations and improves compatibility for modern hints such as <code>list[int]</code> .        |
| Method chaining                                 | <code>sort_values(...).reset_index(...).copy()</code> passes one DataFrame through consecutive transformations.           |
| GroupBy object                                  | A split-apply-combine interface; operations are performed independently inside each <code>cell_id</code> group.           |
| <code>shift</code>                              | Moves values relative to their row positions within each group; negative shift brings future values upward.               |
| Vectorization                                   | <code>dx</code> and <code>dy</code> are calculated for entire Series objects without a Python loop over rows.             |
| <code>list[int]</code>                          | A list whose intended elements are integers; a static documentation/type-checking hint.                                   |
| <code>pd.concat</code>                          | Stacks multiple compatible DataFrames into one table.                                                                     |
| <code>return</code>                             | Sends the completed object back to the calling notebook or function.                                                      |

[↑ Back to Contents](#)

# 8. Phase 3 Notebook and Verified Numerical Results

## 8.1 Exact notebook code

### Repository notebook cell sequence

```
# Cell 1 – Make the project package importable from notebooks/
import _path

# Cell 2 – Load the cleaned full trajectory table
import pandas as pd

tracks_path = "../data/MS01_tracks_clean.csv"
tracks = pd.read_csv(tracks_path)
tracks.head()

# Cell 3 – Inspect shape, columns, and dtypes
tracks.shape, tracks.columns, tracks.dtypes

# Cell 4 – Import reusable Phase 3 functions
from src.steps import compute_steps_multi_tau, summary_by_tau

# Cell 5 – Compute displacement steps at three lags
taus = [1, 2, 4]
steps_all = compute_steps_multi_tau(tracks, taus)
steps_all.head(), steps_all.shape

# Cell 6 – Summarize contributions by lag
summary = summary_by_tau(steps_all)
summary
```

```
# Cell 7 – Descriptive statistics and mean-displacement drift diagnostic
print(steps_all[["dx_um", "dy_um"]].describe())
steps_all.groupby("tau_frames")[["dx_um", "dy_um"]].mean()

# Cell 8 – Histograms for tau = 1
import matplotlib.pyplot as plt
s1 = steps_all[steps_all["tau_frames"] == 1]

plt.hist(s1["dx_um"], bins=50)
plt.xlabel("dx (μm) for tau=1")
plt.ylabel("count")
plt.show()

plt.hist(s1["dy_um"], bins=50)
plt.xlabel("dy (μm) for tau=1")
plt.ylabel("count")
plt.show()

# Cell 9 – Save the combined step table
out_path = "../data/MS01_steps_tau124.csv"
steps_all.to_csv(out_path, index=False)
out_path

# Cell 10 – Save one file per lag
for tau in taus:
    s = steps_all[steps_all["tau_frames"] == tau].copy()
    s.to_csv(f"../data/MS01_steps_tau{tau}.csv", index=False)

# Cell 11 – Save the summary table
summary.to_csv("../data/MS01_steps_summary_tau124.csv", index=False)
```

## 8.2 Step-support summary

| tau_frames | n_steps | n_cells | median_tau_min |
|------------|---------|---------|----------------|
| 1          | 439     | 35      | 20.0           |
| 2          | 404     | 32      | 40.0           |
| 4          | 344     | 26      | 80.0           |

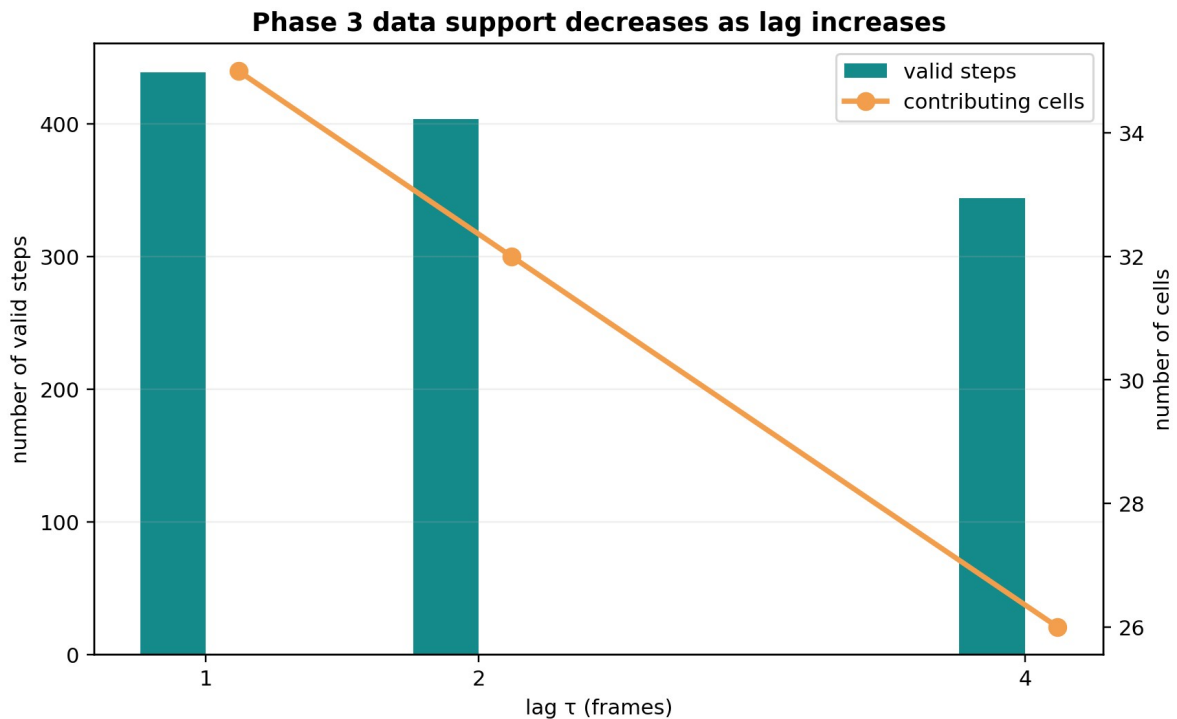


Figure 7. Number of valid displacement rows and contributing tracks at each lag.

The total combined table had 1,187 rows and 9 columns. Declining support with lag is expected: a cell must have a sufficiently long trajectory to provide a displacement over a larger temporal separation.

## 8.3 Descriptive statistics

| Statistic | dx_um      | dy_um      |
|-----------|------------|------------|
| count     | 1187       | 1187       |
| mean      | -0.910636  | -0.337958  |
| std       | 4.522866   | 4.922749   |
| min       | -24.842169 | -34.010622 |
| 25%       | -2.883082  | -1.440636  |
| median    | -0.725372  | 0.000000   |
| 75%       | 1.195453   | 1.292084   |
| max       | 17.578750  | 19.435225  |

The combined mean mixes three lag scales and therefore is not a drift estimate by itself. The per-lag means are more informative. The project record also notes that approximately 18.8% of dy values were exactly zero; this may reflect repeated coordinates, rounding/quantization, or tracking behavior and should be checked directly in the data.

## 8.4 Drift diagnostic

| tau_frames | mean dx ( $\mu\text{m}$ ) | mean dy ( $\mu\text{m}$ ) |
|------------|---------------------------|---------------------------|
| 1          | -0.415209                 | -0.075137                 |
| 2          | -0.831101                 | -0.252589                 |
| 4          | -1.636289                 | -0.773619                 |

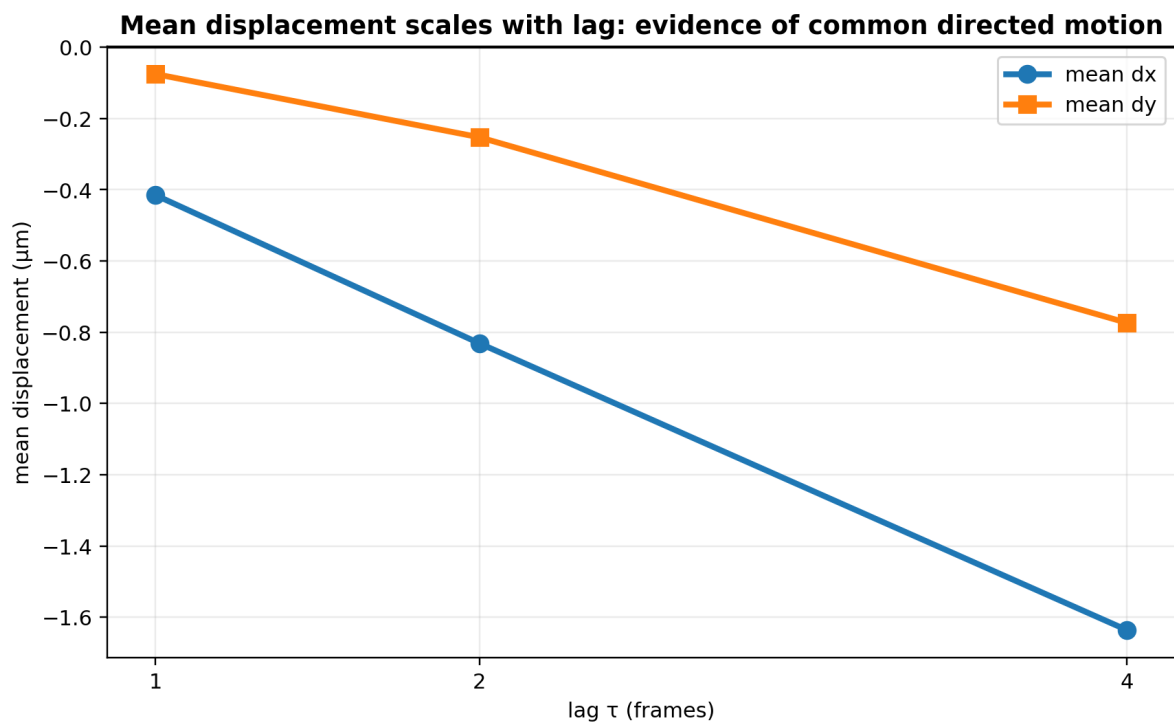


Figure 8. Recorded per-lag mean displacements. The approximately proportional  $x$  trend motivates a drift/common-motion investigation.

The  $x$  component is close to linear in lag: doubling  $\tau$  approximately doubles the mean  $x$  displacement, and quadrupling  $\tau$  approximately quadruples it. This is consistent with a nearly constant common velocity. It does not, by itself, determine whether the source is stage drift, image registration error, bulk sample motion, or genuine collective migration.

## 8.5 Saved artifacts

Files written by the executed Phase 3 notebook in the local working directory

```
data/MS01_steps_tau124.csv
data/MS01_steps_tau1.csv
data/MS01_steps_tau2.csv
```

```
data/MS01_steps_tau4.csv
data/MS01_steps_summary_tau124.csv
```

### Public-repository status

These CSVs are named in the README and written by the notebook, but they are not present in GitHub because `data/*` is ignored.

[↑ Back to Contents](#)

## 9. Reproducibility and Scientific Audit

### 9.1 What is reproducible now

- The Conda environment specification is tracked.
- The reusable displacement module is tracked.
- The Phase 2 and Phase 3 notebooks contain stored code and outputs.
- The import path helpers and smoke test are tracked.
- The TrackMate parameter record is documented in README and the project archive.

### 9.2 Repository gaps and mismatches

| Issue                               | Evidence                                                                                                   | Consequence                                                     | Required fix                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Data absent from GitHub             | <code>data/</code> contains only <code>.keep</code> ; <code>.gitignore</code> ignores <code>data/*</code>  | A new clone cannot run notebooks                                | Add a scripted download/prepare workflow or a small versioned sample plus checksums              |
| Kernel/runtime mismatch             | <code>env.yml</code> specifies Python 3.11; both notebooks store <code>language_info</code> version 3.13.5 | The executed results may not come from the declared environment | Re-run in the registered 3.11 kernel and record <code>sys.executable</code> and package versions |
| Phase 2 save commands commented     | <code>to_csv</code> lines are commented in notebook 02                                                     | Run All does not create inputs needed by notebook 03            | Uncomment or move cleaning into <code>src/io.py</code> plus a reproducible script                |
| README scientific wording           | README equates $p(\Delta)$ vs $p(-\Delta)$ with time irreversibility                                       | Risk of overclaiming                                            | Update terminology and add sequence-level plan                                                   |
| Only import test exists             | <code>test_imports</code> does not check numerical correctness                                             | Regressions in displacement logic can pass                      | Add unit tests with toy trajectories and gaps                                                    |
| Empty figures directory not tracked | <code>.gitignore</code> ignores <code>figures/*</code> and no <code>.keep</code> is present                | Expected path may be absent after clone                         | Track <code>figures/.keep</code> or create directory in scripts                                  |

### 9.3 Algorithmic issues to resolve

- Validate exact frame differences for every step; row shifts are not identical to frame lags when frames are missing.
- Check for duplicate (`cell_id`, `frame`) rows and non-monotonic times.
- Decide whether gap-closed TrackMate links should create long-interval steps or be excluded for exact-lag analyses.
- Validate common motion using framewise medians and, preferably, image registration or static landmarks.
- Use symmetric histogram edges around zero before comparing  $H(\Delta)$  with  $H(-\Delta)$ .
- Bootstrap at the track level, not by independently resampling correlated steps.
- Design the time-reversal control to match the final metric; simply reversing tracks does not necessarily force a KL-type statistic to zero.

### 9.4 Test coverage

#### Current smoke test

```
def test_imports():
    import src.io, src.steps, src.density, src.metrics, src.plots
```

A smoke test answers “Can these modules be imported?” It does not answer “Are the numbers correct?” The next test layer should include a hand-computed two-track dataset, invalid-column tests,  $\tau = 0$  rejection, exact expected row counts, and a missing-frame example.

#### Recommended next unit test (not yet in the repository)

```
def test_single_track_tau1_exact_values():
    tracks = pd.DataFrame({
        "cell_id": [1, 1, 1],
        "frame": [0, 1, 2],
        "t_min": [0.0, 20.0, 40.0],
        "x_um": [0.0, 2.0, 5.0],
        "y_um": [0.0, -1.0, -1.0],
    })
    steps = compute_steps_for_tau(tracks, 1)
    assert steps["dx_um"].tolist() == [2.0, 3.0]
    assert steps["dy_um"].tolist() == [-1.0, 0.0]
    assert steps["tau_min"].tolist() == [20.0, 20.0]
```

[↑ Back to Contents](#)

## 10. Exact Current Status

### 10.1 Completed

- Conceptual orientation and initial project roadmap.
- Conda environment specification and package skeleton.
- Sequence 01 import, calibration, TrackMate detection, and tracking.
- Raw Spots CSV export and pandas cleaning.
- Full and min10 trajectory-table creation in the local project.
- Reusable displacement-step functions for multiple lags.
- Phase 3 execution with stored summary/descriptive outputs.
- Initial common-motion/drift diagnostic.

### 10.2 Not completed

- Independent validation that common motion is imaging drift rather than biological migration.
- Coordinate correction and regenerated corrected steps.
- Versioned Phase 3 analysis of the min10 subset.
- Two-dimensional density estimation with carefully symmetric support.
- A defensibly named single-step asymmetry metric.
- Sequence-level temporal irreversibility.
- Track-level bootstrap confidence intervals.
- Sign-shuffle, reversible-surrogate, and metric-matched time-reversal controls.
- Robustness across bins, KDE bandwidths, lag choices, sequence 02, and tracking source.
- Entropy-production/TUR interpretation.
- Final figures, Methods, and publication package.

### 10.3 Current scientific question

#### Phase 4 gate question

Is the approximately constant negative x motion a non-biological image/stage drift, a whole-sample displacement, a TrackMate artifact, or genuine collective cell migration? The answer determines whether and how coordinates should be corrected.

[↑ Back to Contents](#)

# 11. Updated Roadmap from Phase 4 to Publication

## 11.1 Phase 4 drift validation

14. Re-run Phase 3 in the declared Python 3.11 kernel and verify stored outputs.
15. Audit duplicates, frame gaps, tau\_min distributions, and exact frame differences.
16. Estimate framewise common displacement from  $\tau = 1$  steps using a robust statistic such as the median.
17. Plot framewise vectors, cumulative common motion, and the number of contributing cells per frame.
18. Compare the inferred common motion with direct image registration or stable image features when possible.
19. Decide whether correction is justified and document the biological assumption.
20. Subtract the chosen drift trajectory from coordinates, then regenerate all lags.
21. Verify that mean steps are reduced without creating new artifacts.
22. Repeat for full and min10 datasets.

## 11.2 Displacement asymmetry

After drift validation, estimate the joint distribution of (dx,dy) and its inversion. Use a common symmetric domain and report sensitivity to binning or KDE bandwidth. Name the output according to what it measures.

## 11.3 Proper temporal irreversibility

- Construct ordered pairs ( $\Delta t$ ,  $\Delta t+1$ ) or longer blocks within each trajectory.
- Define the reversed block by reversing order and applying the correct parity transformation to vector/velocity variables.
- Estimate forward and reverse sequence densities or train a cross-validated classifier to distinguish them.
- Use held-out evaluation or bias correction; high-dimensional density ratios are vulnerable to finite-sample bias.

## 11.4 CI, controls, robustness, thermodynamics

| Work package   | Minimum requirement                                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Uncertainty    | Cluster/track bootstrap with 95% confidence intervals                                                                                              |
| Controls       | Sign randomization for single-step asymmetry; reversible surrogate; properly defined time reversal for sequence metric                             |
| Robustness     | full vs min10; lag grid; histogram/KDE settings; sequence 01 vs 02; tracking sensitivity                                                           |
| Thermodynamics | Only interpret a path-probability ratio as an entropy-production bound under clearly stated stochastic assumptions and coarse-graining limitations |
| Reporting      | Save all parameters, seeds, data provenance, figures, and one-command or scripted workflow                                                         |

[↑ Back to Contents](#)

# 12. Resume Protocol for the Next Work Session

## 12.1 Environment and kernel checks

```
conda activate cell-irreversibility
python -c "import sys; print(sys.executable); print(sys.version)"
python -c "import numpy, pandas, matplotlib, sklearn; print(numpy.__version__, pandas.__version__, matplotlib.__version__, sklearn.__version__)"
pytest -q
```

### Expected environment

The executable should be inside `.../envs/cell-irreversibility/` and the Python major/minor version should match `env.yml` (3.11).

## 12.2 Re-run Phase 3

23. Confirm that local raw and clean CSVs exist.
24. Open 03\_compute\_steps\_MSC01.ipynb.
25. Select Python (cell-irreversibility) and verify sys.executable in a dedicated first cell.
26. Restart Kernel and Run All.
27. Confirm 439/404/344 valid steps and 35/32/26 cells for  $\tau = 1/2/4$ .
28. Record any difference instead of forcing the old values.
29. Run the same workflow on MSC01\_tracks\_clean\_min10.csv and save separately.

## 12.3 Begin Phase 4

Next notebook to create

```
notebooks/04_drift_correction_MSC01.ipynb
```

- Load tracks and rebuild raw steps.
- Audit exact lag timing and frame gaps.
- Estimate framewise common motion with uncertainty/support counts.
- Plot and validate the inferred drift.
- Create corrected coordinates only after the validation decision.
- Rebuild steps and compare raw vs corrected diagnostics.

## 12.4 Git checkpoint

Illustrative checkpoint after verified changes

```
git status
git add env.yml src/steps.py tests/ notebooks/ README.md
git commit -m "Audit Phase 3 reproducibility and prepare drift validation"
git push
```

Do not add large raw data blindly. Instead, decide on a data-provenance policy: scripted download, small test fixture, checksums, and documented local paths.

[↑ Back to Contents](#)

# Appendix A. Full Supporting Files

### A1. env.yml

```
name: cell-irreversibility
channels:
  - defaults
dependencies:
  - python=3.11
  - matplotlib
  - pytest
  - numpy
  - pandas
  - scikit-learn
  - jupyterlab
  - ipykernel
```

### A2. .gitignore

```
__pycache__/
.ipynb_checkpoints/
.env
.venv
*.pyc
.DS_Store
data/*
!data/.keep
```

```
figures/*
*.mp4
```

### A3. notebooks/\_path.py

```
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT))
```

### A4. tests/conftest.py

```
import sys, os
ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
```

### A5. tests/test\_imports.py

```
def test_imports():
    import src.io, src.steps, src.density, src.metrics, src.plots
```

### A6. src/\_\_init\_\_.py

```
# makes src a package
```

### A7. src/io.py

```
"""I/O placeholders: later we'll add read_tracks_csv() etc."""
```

### A8. src/density.py

```
"""Density placeholders: later we'll add histogram2d / KDE."""
```

### A9. src/metrics.py

```
"""Metrics placeholders: later we'll implement I_tau, controls, bootstrap."""
```

### A10. src/plots.py

```
"""Plots placeholders: later we'll add plotting helpers."""
```

[↑ Back to Contents](#)

## Appendix B. Data Dictionaries and Artifact Ledger

| Artifact                                     | Role                                       | Public GitHub status | Confidence/status                            |
|----------------------------------------------|--------------------------------------------|----------------------|----------------------------------------------|
| data/raw/Fluo-C2DL-MSD/01/MSD01_allspots.csv | Raw TrackMate Spots export                 | Absent               | Project-record reported; notebook input path |
| data/MSD01_tracks_clean.csv                  | All tracked detections in canonical schema | Absent               | Notebook 03 loaded 474 × 5 table             |
| data/MSD01_tracks_clean_min10.csv            | Tracks with ≥10 frames                     | Absent               | Project-record reported; 18 tracks           |
| data/MSD01_steps_tau124.csv                  | Combined steps for $\tau=1,2,4$            | Absent               | Notebook 03 writes it                        |
| data/MSD01_steps_tau1/2/4.csv                | Per-lag step tables                        | Absent               | Notebook 03 writes them                      |
| data/MSD01_steps_summary_tau124.csv          | Counts and median lag                      | Absent               | Notebook 03 writes it                        |
| notebooks/02_clean_spots_MSD01.ipynb         | Cleaning workflow                          | Present              | Executed; save lines currently commented     |
| notebooks/03_compute_steps_MSD01.ipynb       | Phase 3 workflow                           | Present              | Executed; numeric results stored             |
| src/steps.py                                 | Reusable displacement logic                | Present              | Implemented                                  |
| notebooks/04_drift_correction_MSD01.ip       | Next analysis notebook                     | Absent               | Planned                                      |



| Artifact | Role | Public GitHub status | Confidence/status |
|----------|------|----------------------|-------------------|
| ynb      |      |                      |                   |

[↑ Back to Contents](#)

## Appendix C. Python and Biophysics Glossary

| Term                 | Project-specific meaning                                                                                                                                  |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trajectory           | A time-ordered series of positions for one tracked cell.                                                                                                  |
| Displacement         | A vector difference between two positions.                                                                                                                |
| Lag $\tau$           | The temporal separation used to define a displacement or sequence statistic.                                                                              |
| Drift                | Common apparent motion caused by imaging hardware, registration, sample motion, or other global effects; must be distinguished from collective migration. |
| Detailed balance     | An equilibrium condition relating forward and reverse transitions.                                                                                        |
| Time irreversibility | Statistical distinguishability of a time-ordered process from its correctly defined time reversal.                                                        |
| Entropy production   | A thermodynamic measure of irreversibility; trajectory-level probability ratios can bound or represent it only under explicit models and assumptions.     |
| DataFrame            | A pandas table with labeled rows and columns.                                                                                                             |
| Series               | A one-dimensional labeled pandas object, usually a column.                                                                                                |
| Index                | Labels used to align pandas objects.                                                                                                                      |
| dtype                | The stored data type of a Series or array.                                                                                                                |
| NaN                  | A missing numerical value marker.                                                                                                                         |
| list                 | Ordered mutable collection.                                                                                                                               |
| tuple                | Ordered immutable collection.                                                                                                                             |
| set                  | Unordered collection of unique hashable elements.                                                                                                         |
| dictionary           | Mapping from keys to values.                                                                                                                              |
| function             | Reusable named block of code with inputs and a return value.                                                                                              |
| method               | A function attached to an object, such as <code>df.sort_values()</code> .                                                                                 |
| module               | A Python .py file.                                                                                                                                        |
| package              | A directory of importable Python modules.                                                                                                                 |
| GroupBy              | A pandas object supporting split-apply-combine operations.                                                                                                |
| Vectorization        | Operating on arrays/Series at once instead of explicit row loops.                                                                                         |
| Bootstrap            | Resampling procedure used to estimate uncertainty; here it should resample tracks as clusters.                                                            |
| Smoke test           | A minimal test that verifies basic import/execution, not scientific correctness.                                                                          |
| Unit test            | A focused test of a small function against known expected behavior.                                                                                       |
| Reproducibility      | The ability to regenerate outputs from documented code, environment, parameters, and data provenance.                                                     |

[↑ Back to Contents](#)

## Appendix D. Ready-to-Paste New-Chat Handoff Prompt

### Handoff prompt

I am continuing an in-silico biophysics project on cell-motility time irreversibility. Please read the attached Master Project Handbook before proposing code or analysis.

Treat the sources in this order:

1. The current GitHub repository for code and tracked-file truth:  
`https://github.com/AbolDpirate/cell-irreversibility`
2. Executed notebook outputs for recorded numerical results.
3. The handbook's current-status and audit sections for the stopping point and unresolved issues.

Exact current position:

- Phase 3 core is complete for Fluo-C2DL-MS sequence 01.
- The full clean table had 474 detections across 35 tracks.
- $\tau = 1, 2, 4$  produced 439, 404, and 344 steps from 35, 32, and 26 cells.

- Mean displacement scales with lag, especially in x, suggesting common directed motion.
- Phase 4 drift validation/correction has not yet been executed.

#### Teaching requirements:

- Teach Python, mathematics/statistics, biophysics, scientific computing, reproducibility, testing, and Git/GitHub while advancing the real project.
- Explain Python objects and syntax in depth whenever introduced (lists, tuples, dictionaries, sets, Series, DataFrames, functions, groupby, shift, etc.).
- Keep teaching examples separate from the canonical project code so the workflow remains clear.
- Do not overclaim  $p(\Delta)$  versus  $p(-\Delta)$  as complete temporal irreversibility.
- Do not compute a final  $I_{\tau}$  before validating drift/common motion.

Start by auditing the environment/kernel, reproducing Phase 3, validating exact frame lags and missing-frame behavior, and then designing notebooks/04\_drift\_correction\_MSC01.ipynb.

[↑ Back to Contents](#)

## Appendix E. References and Source Snapshot

[R1] AbolDpirate/cell-irreversibility GitHub repository, main branch, inspected 26 July 2026.

<https://github.com/AbolDpirate/cell-irreversibility>

[R2] Cell Tracking Challenge – 2D+Time datasets and Fluo-C2DL-MSC download page.

<https://celltrackingchallenge.net/2d-datasets/>

[R3] Tinevez J-Y et al. TrackMate: An open and extensible platform for single-particle tracking. *Methods*.

2017;115:80–90. <https://pubmed.ncbi.nlm.nih.gov/27713081/>

[R4] TrackMate LAP trackers documentation. <https://imagej.github.io/plugins/trackmate/trackers/lap-trackers>

[R5] Maška M et al. The Cell Tracking Challenge: 10 years of objective benchmarking. *Nature Methods*. 2023.

<https://www.nature.com/articles/s41592-023-01879-y>

[R6] Seifert U. Stochastic thermodynamics, fluctuation theorems and molecular machines. *Reports on Progress in Physics*. 2012;75:126001.

<https://arxiv.org/abs/1205.4176>

[R7] Gnesotto FS et al. Broken detailed balance and non-equilibrium dynamics in living systems: a review. *Reports on Progress in Physics*. 2018;81:066601.

<https://pubmed.ncbi.nlm.nih.gov/29504517/>

[R8] Roldán É et al. Quantifying entropy production in active fluctuations of the hair-cell bundle from time

irreversibility and uncertainty relations. *New Journal of Physics*. 2021;23:083013. <https://doi.org/10.1088/1367-2630/ac0f18>

Internal project sources used: the 70-page instructions.docx conversation archive, the final Pasted markdown.md status record, and the executed notebook outputs visible in GitHub. The code snapshot and repository audit should be rechecked whenever the main branch changes.

[↑ Back to Contents](#)

## END OF VERIFIED PROJECT RECORD — PHASE 3 SNAPSHOT